

به نام ایزد یکتا

برنامه سازی پیشرفته - 4041

استاد درس : آقای دکتر علی جاویدانی

اعضای تیم دستیاران دانشجویی : [مبینا صفری](#) - [مهدیس یعقوب انصاری](#) - [متین امیرپناه فر](#) -

[شهاب ثمری شمس](#) - [محمد متین سلیمانی](#) - [پارسا دولت آبادی](#) - [نیما مخملی](#)



تمرین دوم | بازی دوز

مهلت تحویل: 1404/08/27 - ساعت 23:59

سلام بچه ها ... 😊

به مینی پروژه ی دومتون خیلی خوش اومدید!

در این پروژه، هدف اصلی آشنایی شما با مفهوم «class» در زبان cpp و نحوه ی استفاده از آن به جای ساختارهای سنتی مانند «struct» است.

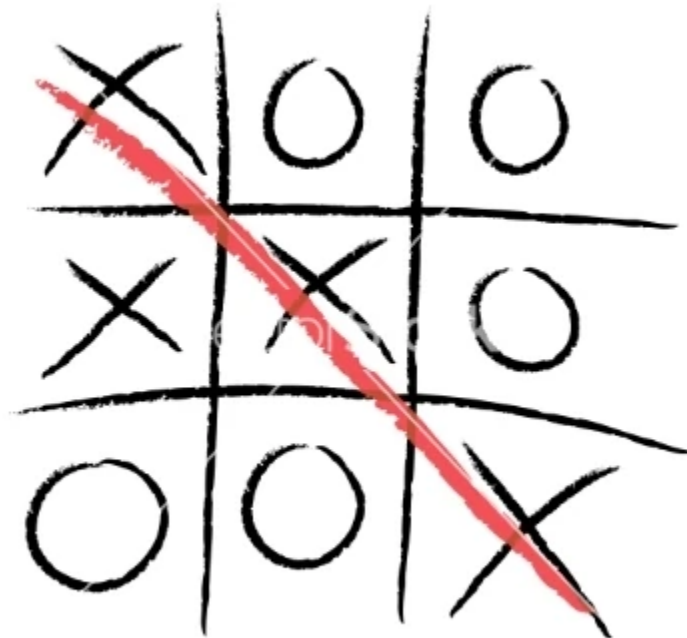
با استفاده از کلاس ها، می توان داده ها و توابع مرتبط با آن ها را در یک قالب منسجم تر سازمان دهی کرد تا کد خوانتر، قابل نگهداری تر و توسعه پذیرتر شود.

در این دوتا مینی پروژه، شما با پیاده سازی بازی ساده ی **دوز و مسابقه ی خرگوش و لاک پشت** تمرین می کنید چطوری کلاس را برای ساخت یک برنامه ی کاربردی به کار بگیرید. تمرکز اصلی این پروژه بر درک مفاهیم پایه ای کلاس ها هست :

- Class Collaboration
- Access Specifiers
- Member Variables
- Member Function

توضیح کلی پروژه اول

در این پروژه شما باید بازی کلاسیک دوز را برای دو بازیکن پیاده سازی کنید. بازی بر روی یک جدول 3×3 انجام می‌شود و هر بازیکن به نوبت یک خانه‌ی خالی را با علامت خود (X یا O) پر می‌کند. هدف هر بازیکن این است که سه علامت خود را به صورت افقی، عمودی یا قطری در جدول قرار دهد تا برنده شود. در صورتی که تمامی خانه‌ها پر شوند و هیچ بازیکنی نتواند این شرط را رعایت کند، بازی مساوی اعلام می‌شود. برنامه با نمایش وضعیت فعلی جدول پس از هر حرکت، نوبت بازیکنان را کنترل کرده و نتیجه‌ی نهایی را اعلام می‌کند.



قوانین بازی دوز

قوانین زیر نحوه‌ی بازی دوز را مشخص می‌کنند:

- هر بازیکن در هر نوبت تنها می‌تواند یک علامت X یا O در جدول 3×3 قرار دهد.
- بازیکنان به صورت نوبتی بازی می‌کنند تا بازیکنی برنده شود یا بازی مساوی شود.
- برای برنده شدن، بازیکن باید یک خط افقی، عمودی یا قطری شامل سه علامت یکسان بسازد.
- اگر همه خانه‌ها با X یا O پر شوند ولی هیچ خطی شکل نگیرد، بازی مساوی (Draw) اعلام می‌شود.

ویژگی‌های بازی دوز

این بازی امکانات زیر را ارائه می‌دهد:

- این بازی روی یک جدول 3×3 طراحی شده است.
- این بازی برای دو بازیکن طراحی شده است.
- هر بازیکن می‌تواند یکی از حروف X یا O را انتخاب کند.
- هر دو بازیکن به نوبت فرصت بازی خواهند داشت.

اجزای بازی

۱. صفحه‌ی بازی (Game Board)

صفحه‌ی بازی توسط کلاس Board مدیریت می‌شود که شامل موارد زیر است:

- یک شبکه‌ی 3×3 از نوع کاراکتر برای نمایش صفحه‌ی بازی
- یک شمارنده برای پیگیری خانه‌های پر شده

۲. مدیریت بازیکن (Player Management)

بازیکنان توسط کلاس Player نمایش داده می‌شوند که اطلاعات زیر را ذخیره می‌کند:

- علامت بازیکن (X یا O)
- نام بازیکن

۳. حرکات بازیکن (Movement Of Player)

کلاس Board شامل متدهایی برای مدیریت حرکات بازیکن است:

- drawBoard() برای نمایش وضعیت فعلی صفحه
- isValidMove() برای بررسی صحت حرکت
- makeMove() برای به‌روزرسانی صفحه با حرکت بازیکن

چگونه بررسی کنیم که ورودی معتبر است یا خیر؟

- **ورودی معتبر:** اگر خانه خالی باشد و در محدوده قرار داشته باشد (۰-۲ برای چک کردن در آرایه، ۱-۳ برای ورودی کاربر)
- **ورودی نامعتبر:** اگر خانه قبلاً با حرف دیگری پر شده باشد یا خارج از محدوده باشد.

۴. منطق بازی (Game Logic)

کلاس Dooz منطق کلی بازی را مدیریت می‌کند:

- مدیریت نوبت بازیکنان
- پردازش ورودی کاربر
- بررسی شرایط برد یا مساوی

۵. برد، باخت یا مساوی (Win, Lose or Draw)

کلاس Board شامل متدهایی برای بررسی وضعیت بازی است:

- `checkWin()` برای تعیین اینکه آیا بازیکنی برنده شده است
- `isFull()` برای بررسی پر بودن صفحه (مساوی)

شرط مساوی در متد `play()` کلاس Dooz بررسی می‌شود، وقتی که صفحه کامل شده و هیچ بازیکنی برنده نشده باشد.

ساختار کلی

۱. کلاس **Player**

۲. کلاس **Board**

۳. کلاس **Dooz**

توضیح کلی پروژه دوم

در این پروژه، شما باید مسابقه‌ی کلاسیک خرگوش و لاکپشت را شبیه‌سازی کنید. در این بازی، دو حیوان در یک مسیر فرضی با طول ثابت (مثلاً ۷۰ خانه) با یکدیگر مسابقه می‌دهند. حرکت هر حیوان به صورت تصادفی (Random) و بر اساس قوانین مشخص انجام می‌شود. در هر لحظه، جایگاه خرگوش و لاکپشت در مسیر نمایش داده می‌شود و برنامه تا زمانی ادامه می‌یابد که یکی از آن‌ها به خط پایان برسد. در پایان، برنامه برنده را اعلام می‌کند یا در صورت رسیدن همزمان هر دو، بازی را مساوی اعلام می‌کند.



قوانین مسابقه‌ی خرگوش و لاک‌پشت

حرکت هر حیوان در هر «تیک زمانی» (Iteration) طبق احتمال‌های مشخص انجام می‌شود:

لاک‌پشت (Tortoise) 🐢

- حرکت سریع (Fast Plod): با احتمال ۵۰٪ → حرکت ۳ خانه به جلو
- لغزش (Slip): با احتمال ۲۰٪ → حرکت ۶ خانه به عقب
- حرکت آهسته (Slow Plod): با احتمال ۳۰٪ → حرکت ۱ خانه به جلو

خرگوش (Hare) 🐰

- خواب (Sleep): با احتمال ۲۰٪ → بدون حرکت
- جهش بلند (Big Hop): با احتمال ۲۰٪ → حرکت ۹ خانه به جلو
- سر خوردن بلند (Big Slip): با احتمال ۱۰٪ → حرکت ۱۲ خانه به عقب
- جهش کوتاه (Small Hop): با احتمال ۳۰٪ → حرکت ۱ خانه به جلو
- سر خوردن کوتاه (Small Slip): با احتمال ۲۰٪ → حرکت ۲ خانه به عقب

در صورتی که موقعیت هر حیوان از مسیر مسابقه خارج شود. (کمتر از خانه‌ی ۱ یا بیشتر از خانه‌ی ۷۰)، باید موقعیت آن به نزدیکترین حد مجاز اصلاح شود.

اگر هر دو حیوان در یک مکان قرار بگیرند، در خروجی عبارت "!!!OUCH" نمایش داده می‌شود.

ویژگی‌های بازی خرگوش و لاکپشت

این بازی امکانات زیر را ارائه می‌دهد:

- حرکت تصادفی دو بازیکن (خرگوش و لاکپشت) بر اساس احتمال‌های تعیین‌شده
- نمایش مسیر مسابقه در هر مرحله
- اعلام برنده یا تساوی در پایان مسابقه
- استفاده از توابع مجزا برای مدیریت رفتار هر حیوان

اجزای اصلی پروژه

۱. مسیر مسابقه (Race Track)

- نمایش مسیر مسابقه به صورت آرایه‌ای از کاراکترها (مثلاً طول ۷۰)
- به‌روزرسانی موقعیت خرگوش و لاکپشت در هر تکرار
- نمایش "T" برای لاکپشت، "H" برای خرگوش و "OUCH!!!" در صورت هم‌مکانی

۲. حیوانات (Animals)

هر حیوان توسط یک کلاس مجزا نمایش داده می‌شود.

کلاس Tortoise

ویژگی‌ها:

- موقعیت فعلی (Position)
- رفتارها:
- move() برای تعیین حرکت تصادفی طبق قوانین لاکپشت

کلاس Hare

ویژگی‌ها:

- موقعیت فعلی (Position)
- رفتارها:
- move() برای تعیین حرکت تصادفی طبق قوانین خرگوش

۳. مدیریت مسابقه (Race Management)

کلاس Race مسئول کنترل کل فرآیند مسابقه است. وظایف آن شامل:

- مقداردهی اولیه موقعیت‌ها
- اجرای حلقه‌ی اصلی مسابقه تا زمان رسیدن یکی از حیوان‌ها به خط پایان
- چاپ مسیر در هر مرحله
- اعلام نتیجه (خرگوش، لاکپشت یا تساوی)

منطق بازی (Game Logic)

1. در هر مرحله، توابع حرکت خرگوش و لاکپشت فراخوانی می‌شوند.
2. موقعیت هر حیوان به‌روزرسانی شده و اطمینان حاصل می‌شود که در محدوده‌ی مسیر قرار دارند.
3. مسیر با کاراکترهای جدید چاپ می‌شود.
4. اگر موقعیت یکی از حیوان‌ها ≤ 70 شود، بازی خاتمه می‌یابد.
5. در صورت برابری در خط پایان، نتیجه مساوی اعلام می‌شود.

شرایط پایان مسابقه

بازی در یکی از حالت‌های زیر خاتمه می‌یابد:

- اگر لاکپشت زودتر به خط پایان برسد → "!!!!TORTOISE WINS"
- اگر خرگوش زودتر برسد → "!!!!HARE WINS"
- اگر هر دو همزمان برسند → "!!!!IT'S A TIE"

ساختار کلی برنامه

۱. کلاس Tortoise

۲. کلاس Hare

۳. کلاس Race

📁 نحوه تحویل

- تمامی فایل‌های مرتبط با تمرین، شامل سورس‌کد و هرگونه مستندات یا توضیحات تکمیلی، باید در یک پوشه با نام زیر قرار گیرند:

HW2-StudentNumber

- این پوشه را فشرده کرده و با فرمت zip ارسال نمایید.
- ارسال فایل تنها از طریق بخش "تمرین دوم" در کلاس درس، در سامانه کونرا صورت می‌گیرد.
- **توجه داشته باشید که استفاده از Git, GitHub در این پروژه الزامیست.**
- می‌توانید مینی پروژه ها را با هدر فایل ها هم پیاده سازی کنید.

لطفاً پیش از اتمام مهلت مقرر، از آپلود صحیح فایل اطمینان حاصل فرمایید.

زندگی نیست بجز نم نم باران بهار
زندگی نیست بجز دیدن یار
زندگی نیست بجز عشق، بجز حرف محبت به کسی
ورنه هر خار و خسی زندگی کرده بسی
(س. سپهری)